

The background is a blurred image of a code editor showing SVG code. Visible code includes:

```
<defs>  
  <linearGradient x1="100%" y1="0%" x2="0%" y2="100%">  
    <stop stop-color="#06101F" offset="0%" />  
    <stop stop-color="#1D304B" offset="100%" />  
  </linearGradient>  
</defs>  
<rect width="800" height="450" rx="8" fill="url(#gradient)" />  
</svg>  
<div>  
  <div>  
    <div>  
      <div>  
        <div>  
          <div>  
            <div>  
              <div>  
                <div>  
                  <div>  
                    <div>  
                      <div>  
                        <div>  
                          <div>  
                        </div>  
                      </div>  
                    </div>  
                  </div>  
                </div>  
              </div>  
            </div>  
          </div>  
        </div>  
      </div>  
    </div>  
  </div>  
</div>
```

express

Database Integration

Database integration

- Adding the capability to connect databases to Express apps is just a matter of loading an appropriate Node.js driver for the database in your app.
- This section briefly explains how to add and use mySql Node.js module for database systems in your Express app

```
$ npm install mysql2
```



express

Database Integration

step by step example

Step 1: Create Node.js application

- Open terminal/console, then create a folder for our application:

```
mkdir DatabaseIntegration  
cd DatabaseIntegration
```

- Initialize the Node.js application with a package.json file:

```
npm init -y
```

- Next, we need to install necessary modules: express, mysql and body-parser. Run the command:

```
npm i express mysql2 body-parser
```

Step 2: Setup Express web server

- In the root folder, we create a new file named index.js:

```
var express=require ("express");
var bodyParser=require ("body-parser");
const app = express();
const port = 3000;

// parse requests of content-type: application/json
app.use(bodyParser.json());
// parse requests of content-type: application/x-www-form-urlencoded
app.use(bodyParser.urlencoded({ extended: true }));

// set port, listen for requests
app.listen(port, () => {
  console.log(`Server is running on port ${port}.`);
});
```

Open MySql command line

Before connecting Node.js Application with MySQL, we need to create a table.

1. Open Command Prompt.
2. Navigate to the bin folder.

For example: `cd C:\Program Files\MySQL\MySQL Server 8.0\bin`

1. Run the **mysql -u root -p** command.
2. Enter the password you downloaded mysql with.

Step 3: Create MySQL table

- Run the SQL script (in MySQL command line or in MySQL workbench) below to create customers' table:

```
use mysql;
```

```
CREATE TABLE IF NOT EXISTS `customers` (  
  id int(11) NOT NULL PRIMARY KEY AUTO_INCREMENT,  
  email varchar(255) NOT NULL,  
  name varchar(255) NOT NULL  
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert into customers (name, email) VALUES ("bob dylan", "bobyd@gmail.com");
```

Step 4: Configure & Connect to MySQL DB

- create db.config.js file inside the root folder with:

```
module.exports = {  
  HOST: "localhost",  
  USER: "root",  
  PASSWORD: "*****", //the password to your MySql Databases  
  DB: "mysql"  
};
```


Step 4: Configure & Connect to MySQL DB

- Now create a database connection that uses configuration above.
- The file for connection is *db.js* saved in the root folder as well.

```
const mysql = require("mysql2");
const dbConfig = require("../db.config.js");

// Create a connection to the database
const connection = mysql.createConnection({
  host: dbConfig.HOST,
  user: dbConfig.USER,
  password: dbConfig.PASSWORD,
  database: dbConfig.DB
});

// open the MySQL connection
connection.connect(error => {
  if (error) throw error;
  console.log("Successfully connected to the database.");
});

module.exports = connection;
```

Step 5: Define routes for CRUD operations

- What is CRUD operations?
 - Create
 - Read
 - Update
 - Delete

Step 5: Define routes for CRUD operations

- In index.js define DB connection and use it to write CRUD functions:
 1. get all Customers
 2. create a new Customer
 3. find a Customer by name
 4. update a Customer by name
 5. remove a Customer
 6. remove all Customers
- Lets start with “get all Customers”

“get all Customers” in one block in index.js

- In index.js add a route for “get all Customers”

```
// Define DB connection
var sql = require('./db')

// Create a route for getting all customers
app.get("/customers", function(req, res){
  sql.query("SELECT * FROM customers", (err, mysqlres) => {
    if (err) {
      console.log("error: ", err);
      res.status(400).send( "error in getting all customers: " + err);
      return;
    }

    console.log("got all customers...");
    res.send(mysqlres);
    return;
  });
});
```

“Create new customer” split into 2 files

- Create a new file named CRUD_functions.js (on root folder)
- In this file define the following function:

```
const sql = require("../db.js");
const createNewCustomer = function(req,res){
  // Validate request
  if (!req.body) {
    res.status(400).send({
      message: "Content can not be empty!"
    });
    return;
  }
  const newCustomer = {
    "email": req.body.email,
    "name": req.body.name
  };
  sql.query("INSERT INTO customers SET ?", newCustomer, (err, mysqlres) => {
    if (err) {
      console.log("error: ", err);
      res.status(400).send( "error in creating customer: " + err);
      return;
    }
    console.log("created customer: ", { id: mysqlres.insertId, ...newCustomer });
    res.send("new customer created successfully");
    return;
  });
};
module.exports = {createNewCustomer};
```

“Create new customer” split into 2 files

- Inside index.js define a route using the handler as the function from CRUD_functions.js:
1. At the top of the file, link the CRUD_functions.js by adding:

```
const CRUD_operations = require("./CRUD_functions.js");
```

2. Then define route for post request at path /newCustomer:

```
// Create a new Customer  
app.post("/CreateNewCustomer", CRUD_operations.createNewCustomer);
```

“Create new customer” client side

- Create an HTML file in the root folder called “Create_new_customer.html”

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Form connected to DB</title>
</head>
<body>
  <form action="/CreateNewCustomer" method="POST">
    <label for="name">Your Name</label>
    <input type="text" id="name" name="name"><br>
    <label for="email">Your email</label>
    <input type="text" id="email" name="email"><br>
    <input type="submit" value="Submit">

  </form>
</body>
</html>
```



```
<style>
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;}
form {
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  width: 300px;}
h1 {
  text-align: center;
  color: #333;}
label {
  display: block;
  margin-bottom: 8px;
  color: #555;}
input {
  width: 100%;
  padding: 8px;
  margin-bottom: 16px;
  border: 1px solid #ccc;
  border-radius: 4px;}
input[type="submit"] {
  background-color: #4caf50;
  color: #fff;
  cursor: pointer;}
input[type="submit"]:hover {
  background-color: #45a049;}
</style>
```

Serve html form as homepage

- Meanwhile, save the form as the index file for path “/” in the root folder
- Serve this file for this path by creating the proper route in index.js file using res.sendFile().
- To index.js add at the top:

```
var path = require('path');
```

- And add the route for the get request for '/':

```
app.get('/', (req,res) => {  
  res.sendFile(path.join(__dirname, "Create_new_customer.html"))  
});
```

Check yourself

- Open your browser to <http://localhost:3000/>
- Submit a form
- You should see a message:
“new customer created successfully”

“find a Customer by name”

- In CRUD_functions.js define the function for finding a customer by name
- We will use the method “get”

```
const findCustomer = (req,res)=>{
  const Customer = req.query.FindName;
  console.log(Customer);
  //insert query
  sql.query("SELECT * FROM customers where name like ?", Customer + "%", (err, results)=>{
    if (err) {
      console.log("error is: " + err);
      res.status(400).send( "error in finding customer " + err);
      return;
    }
    // if not query error
    console.log("new customer created ", createNewCustomer);
    res.send(results)
    //res.send({message: "new customer created successfully"});
    return;
  });
};
module.exports = {FindCustomer};
```

“find a Customer by name”

- In index.js define the appropriate route:

```
// get customer by name  
app.get("/FindCustomersByName", CRUD_operations.findCustomer);
```

“find a Customer by name” – client side

- Go to: <http://localhost:8080/FindCustomersByName?FindName=>
- At the end add the name you are looking for (that's the query parameters)